Metro Navigator & GPS Logger — High-Level Design (HLD)

Offline underground positioning for metro travel, and a simple GPS logger

24 September 2026

Document control

1. Executive summary
2. Scope
3. Users and main use cases
4. Operating scenarios (GPS × network)
5. System context
6. Architecture overview
7. Main components
8. How a journey works (data flow)
9. Position sources and confidence
10. Technology stack
11. Data and storage
12. Security, privacy and permissions
13. Non-functional characteristics
14. Deployment
15. Limitations and risks (read before production roll-out)
16. Glossary

Document control

Item	Value
Document	High-Level Design (HLD)
Product	Metro Navigator (`:app`) and GPS Logger (`:gpslogger`), shared library `:core`
Version	1.0
Date	24 September 2026
Status	Code complete, unit/simulation tested, on-device smoke tested. **Pending: field validation on a real underground metro ride.**
Repository	`TestGPSLocation/` (Android Studio / Gradle project)
Companion document	`02_Technical_Design.md` (detailed design, every algorithm, constant, edge case and test)

1. Executive summary

The project delivers **two Android applications** built from one code base:

1. **Metro Navigator** — a navigation and telemetry app that keeps giving the user a position when **GPS and mobile network are both unavailable**, such as inside metro tunnels. While online it silently downloads the metro lines around the user (OpenStreetMap data) and stores them on the phone. Underground it combines the stored track, the phone's motion sensors and automatic station-stop detection to estimate where the train is, which station comes next, distance and time to it, and how many stops remain. Every position is written to a daily log file and shown in an on-screen list.
2. **GPS Logger** — a deliberately simple app: every GPS fix (time, latitude, longitude, accuracy) is written to a text file and shown in a list.

Does it work in the target scenario? Yes, by design and as verified in simulation: a high-speed train (tested at 80 km/h and 108 km/h) that enters a tunnel at speed, with no GPS and no network underground, is tracked along the line; every station stop is recognised in order and the position is snapped exactly to the platform at each stop. Between stations the estimate is approximate (simulated worst case ≈ 250–450 m before the next stop corrects it). Underground the app provides an **estimated** position, not GPS data: satellite signals do not reach tunnels, and no phone can produce real GPS fixes there. Two preconditions: the metro line must have been stored while online (automatic within 2 km, or "Download whole city"), and the result must be confirmed by a real ride (section 15).

2. Scope

In scope

- Positioning in all combinations of GPS quality (good / weak / none) and network quality (good / weak / none).
- Metro-specific underground navigation: line, previous/next station, distance and ETA to next station, destination and stops remaining, direction of travel, interchange handling.
- Offline storage of metro line geometry and stations.

- Continuous background operation (screen off) with a foreground service.
- Per-day telemetry log files in the public *Download* folder; on-screen history.
- Self-diagnosis: warnings with one-tap fixes (location off, approximate permission only, battery optimisation, notifications off, missing sensors, mock locations, storage errors).
- A separate minimal GPS logging app.

Out of scope (current version)

- Map display (the apps show coordinates, stations and lists, not map tiles).
- Journey planning across multiple lines / timetables.
- Server backend, user accounts, cloud upload of logs.
- iOS.

3. Users and main use cases

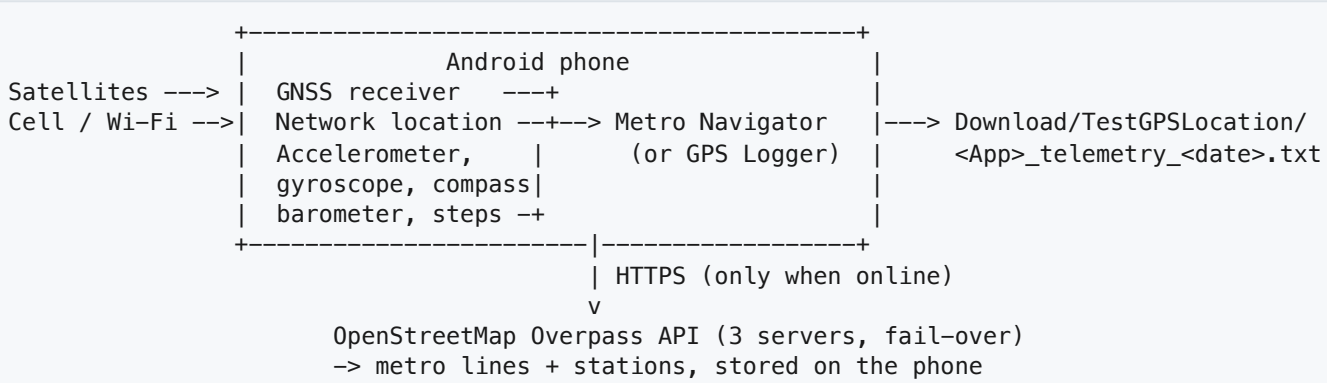
#	Use case	App
UC1	Commuter rides an underground metro and wants to know the current/next station and distance without GPS or network	Metro Navigator
UC2	Tester records a trip (GPS + estimated positions + events) for later analysis	Metro Navigator (Start Logging)
UC3	User boards at an underground station (GPS already lost before boarding)	Metro Navigator
UC4	User changes line at an interchange while underground	Metro Navigator
UC5	User wants a plain record of GPS latitude/longitude in a file and on screen	GPS Logger
UC6	Before travelling, user pre-downloads all lines of the city for offline use	Metro Navigator (“Download whole city”)

4. Operating scenarios (GPS × network)

GPS \ Network	Good network	Weak / unstable network	No network
Good GPS	Satellite position. Nearby metro lines downloaded & stored automatically.	Satellite position. Downloads retried with back-off (30 s → 10 min); offline lookups never wait for downloads.	Satellite position (GPS does not need network; first fix can be slower). Stored lines used; download retried the moment a network appears.
Weak / intermittent GPS (tunnel portals, tall buildings)	Implausible jumps rejected; poor fixes used only as soft corrections; after an outage GPS must be confirmed (2 consistent fixes or ≤ 15 m accuracy) before it is trusted again.	Same.	Same.

GPS \ Network	Good network	Weak / unstable network	No network
No GPS (underground)	**Track navigation + network corrections**: cell/Wi-Fi location nudges the along-track estimate through a Kalman update; outliers rejected.	Same, with fewer corrections.	**Pure track navigation**: motion-sensor speed model + station-stop snapping. Error grows between stations and resets at every stop.
No GPS, no stored line	Network location only (label NET). Lines download automatically when possible.	Network location when available.	Dead reckoning from last heading and speed (label DR), low confidence, clearly flagged.

5. System context



6. Architecture overview

Three Gradle modules:

Module	Type	Responsibility
`:core`	Android library	Shared models (`GpsPoint`), geodesy (`GeoToolbox`), network quality (`NetworkStatus`), robust file logging (`TelemetryLogger`).
`:app`	Android application **Metro Navigator** (`com.example.testgpslocation`)	Sensors, navigation engine, metro data service, foreground logging service, UI.
`:gpslogger`	Android application **GPS Logger** (`com.example.gpslogger`)	Foreground service logging GPS fixes, list UI.

Layers inside Metro Navigator:

```
UI layer          MainActivity  (status, stations, warnings, history list, buttons)
                       |  observes StateFlows
Orchestration     NavigationController (process-wide singleton, shared by UI + service)
                  |            |                    |
Engine      UndergroundNavigator   MetroRouteService   (MetroRoute, TrackStitcher)
(pure logic:  position, track,     offline cache,
 unit-tested) stations, errors     downloads, back-off
                  ^
Sensors     SensorHub  (GNSS, fused/network location, IMU, barometer, step detector)
Background  LocationTrackerService (foreground service, wake lock, file logging)
Shared      :core  TelemetryLogger, NetworkStatus, GeoToolbox, GpsPoint
```

Design principles:

- **Pure, testable engine.** `UndergroundNavigator`, `MetroRoute`, `TrackStitcher` contain no Android types; all algorithms are unit- and simulation-tested on the JVM.
- **Offline first.** Navigation reads only local data. The network is used solely to refresh line data, never on the navigation path.
- **Honest output.** Every position carries its source (GPS / NET / TRACK / DR) and an estimated error (±metres) that grows without GPS and shrinks with every correction.
- **Fail soft.** Missing sensors, missing Google Play services, storage failures, server outages and permission gaps degrade features with a visible warning; they do not crash.

7. Main components

Component	What it does
SensorHub	Collects satellite fixes (GNSS provider), network/fused fixes, satellite counts, and IMU data. Produces 10 Hz motion samples (world-frame acceleration; state = stationary / walking / moving). Declares “GPS lost” after 6 s without a satellite fix. Falls back to raw accelerometer or platform network provider where needed.
UndergroundNavigator	The positioning brain: jump filtering, GPS-recovery confirmation, track locking, dead-reckoning over the detection gap, speed model, station hold & snap, network Kalman correction, direction and interchange detection, cruise-speed learning, uncertainty tracking, navigation outputs.
MetroRoute / TrackStitcher	Line geometry: orders and joins track segments, positions stations along the track, projects a coordinate onto the track, returns point/bearing at a distance.
MetroRouteService	Downloads metro lines from Overpass (3 servers), parses and stores them atomically, remembers downloaded areas for 7 days, answers offline “which lines are near me”.
NavigationController	Wires sensors → navigator → UI/service; schedules lookups and downloads (back-off, retry on network return); publishes state, history, events, line status, satellites.
LocationTrackerService	Foreground service (type *location*) with partial wake lock so navigation continues with the screen off; writes every position and event to the log; live notification.

Component	What it does
MainActivity	Displays position, source, accuracy, satellites, line, stations, ETA, destination picker, flip-direction, error ±m, warnings with fixes, line-data status and buttons, history.
TelemetryLogger (`:core`)	Asynchronous, ordered, crash-safe logging to one file per day in `Download/TestGPSLocation/` (MediaStore on Android 10+), with fall-back to app storage and error reporting.
GPS Logger app	Foreground service → `LocationLog` → file + list; restores list from file after restart.

8. How a journey works (data flow)

1. **Above ground, online.** Satellite fixes give the position. Within 10 s the controller checks the stored lines near the user; if the area (2 km) has not been downloaded in the last 7 days, the lines are downloaded from OpenStreetMap and stored.
2. **Approaching / on the line.** When the user is within 60 m (+ GPS accuracy) of a stored track, the navigator locks to it: distance along the line, direction of travel (from GPS bearing) and speed are tracked continuously. Cruise speed is learnt from GPS.
3. **Entering the tunnel.** No satellite fix for 6 s → "GPS lost". The navigator carries the position forward over those 6 s at the last speed, so the estimate does not lag.
4. **Underground.** Every 100 ms the speed model advances the position along the track. When the train stops for 5 s near the expected station, the position snaps to that station and the error resets; the time taken for the segment refines the cruise speed. If network location is available it corrects the estimate; wild values are rejected.
5. **Boarding underground / interchanges.** When the train pulls out, the direction of the acceleration shows the travel direction and, at interchanges, which line the train is on. The user can also choose a destination or press *Flip*.
6. **Exiting.** When GPS returns and is confirmed, the app reports how far off the estimate was (recovery error) and resumes satellite positioning.

9. Position sources and confidence

Label	Meaning	Typical error
GPS	Satellite fix (accuracy ≤ 50 m, plausibility-checked)	3–15 m
NET	Cell / Wi-Fi location, used when there is no satellite fix	20–500 m
TRACK	Estimated, constrained to the metro track	0–40 m at a station stop; grows between stations (sim. ≤ 450 m)
DR	Dead reckoning without a track	grows quickly; low confidence

Confidence shown to the user is derived from the estimated error: **HIGH** ≤ 30 m, **MEDIUM** ≤ 100 m, **LOW** ≤ 300 m, **VERY LOW** above.

10. Technology stack

Area	Technology (version)	Used for
Language	Kotlin 1.9.0 (JVM target 17)	All application code
Concurrency	Kotlin Coroutines 1.7.3 (`kotlinx-coroutines-android`), `StateFlow`, `SharedFlow`, `Mutex`	Background I/O, UI state streams, safe concurrent cache access
Build	Gradle 8.8 (wrapper), Android Gradle Plugin 8.5.0, Kotlin DSL (`*.gradle.kts`)	Build, test, lint
Android SDK	compileSdk 34, targetSdk 34 (Android 14), **minSdk 24 (Android 7.0)**	Platform APIs
Java	Source/target compatibility 17	Bytecode level
Location	Android `LocationManager` (GNSS provider, network provider), `GnssStatus`; Google Play services Location 21.3.0 (Fused Location Provider, Settings Client)	Satellite fixes, network fixes, satellite counts, “turn on location” dialog
Sensors	Android `SensorManager`: linear acceleration (fallback: accelerometer), rotation vector, gyroscope, barometer, step detector	Motion, orientation, stop detection, walking detection, pressure change
Background	Foreground Service (type `location`), `PowerManager` partial wake lock, notifications (`NotificationCompat`)	Screen-off navigation and logging
Storage	`MediaStore.Downloads` (Android 10+), public Download folder (Android ≤ 9), app-private files, JSON (`org.json`)	Log files; offline metro line cache
Networking	`HttpURLConnection` (HTTPS POST), `ConnectivityManager` network callback & capabilities	Overpass downloads; network quality; retry on reconnect
Map data	OpenStreetMap via Overpass API (overpass-api.de, overpass.kumi.systems, overpass.private.coffee)	Metro line geometry and station names (ODbL licence)
UI	AndroidX AppCompat 1.6.1, Material Components 1.8.0, RecyclerView 1.3.1, CardView 1.0.0, Lifecycle runtime KTX 2.6.2 (`repeatOnLifecycle`)	Screens, lists, lifecycle-aware collection
Testing	JUnit 4.13.2 (JVM unit tests), deterministic ride simulator, Android Lint, on-device smoke tests via ADB	Verification
Tools	Android Studio, ADB, Git	Development
Documentation	Markdown → pandoc → DOCX; HTML → Chrome headless → PDF	These documents

11. Data and storage

Data	Location	Retention
Telemetry log (Metro Navigator)	`Download/TestGPSLocation/Metro_Navigator_telemetry_<yyyy-MM-dd>.txt`	Kept until the user deletes it; one file per day
GPS log (GPS Logger)	`Download/TestGPSLocation/GPS_Logger_telemetry_<yyyy-MM-dd>.txt`	Same
Offline metro lines	App-private `files/metro_routes/<id>.json`	Kept; refreshed when an area is re-downloaded; not purged by the OS cache cleaner
Downloaded-area memory	App-private `files/metro_routes/downloaded_areas.json`	Entries expire after 7 days
On-screen history	Memory (last 500 entries)	Until the app process ends

12. Security, privacy and permissions

- Location data never leaves the phone. The only network traffic is the **anonymous** Overpass query (a circle around the user's position) to download metro lines.
- Logs are plain text in the user's Download folder; the user controls sharing and deletion.
- No accounts, analytics or third-party tracking.

Permission	Why	If denied
Precise location (`ACCESS_FINE_LOCATION`)	Satellite fixes	GPS unavailable; warning with *Fix*
Approximate location (`ACCESS_COARSE_LOCATION`)	Network fixes	No positioning
Physical activity (`ACTIVITY_RECOGNITION`, Android 10+)	Step detector (walking vs train)	High-vibration fallback only; warning
Notifications (`POST_NOTIFICATIONS`, Android 13+)	Show the running-service notification	Service still runs; warning
Foreground service (+ `_LOCATION`), wake lock	Background navigation / logging	— (install-time)
Internet, network state	Download lines; network quality	Offline only
Storage write (Android ≤ 9 only)	Public Download folder	Log falls back to app folder

13. Non-functional characteristics

Quality	Target / result
Station recognition underground	100 % of stops in all simulated rides (6 scenarios + 20 random seeds)

Quality	Target / result
Error at a station stop	0–5 m in simulation (snapped to the station position)
Error between stations (no GPS, no network)	≈ 320–440 m worst case in simulation; less with network corrections (≈ 260 m)
Speed range	Up to 40 m/s (144 km/h)
Start-up	Position from network within seconds; satellites typically within 5–30 s outdoors
CPU	Motion processing at 10 Hz; UI updates ≤ 4 Hz; track projection only on fixes
Battery	GNSS 1 Hz + IMU 50 Hz + wake lock while logging. Expected continuous use of several % per hour (device dependent); stop logging when not needed
Robustness	No crash paths identified in: permission denial, location off, no Play services, no GNSS chip, missing sensors, server errors/timeouts, corrupt cache, storage full, process restart
Compatibility	Android 7.0 (API 24) to Android 14 (API 34) target; on-device tested on Samsung Galaxy S24+ (SM-S9260), Android 14

14. Deployment

- Build: `./gradlew assembleDebug` produces `app/build/outputs/apk/debug/app-debug.apk` and `gpslogger/build/outputs/apk/debug/gpslogger-debug.apk`.
- Install: `adb install -r <apk>` or Android Studio *Run*.
- Release: configure a signing key in Gradle (`signingConfigs`), enable R8 minification if desired, then `./gradlew assembleRelease` / `bundleRelease` for Google Play.
- Google Play note: a foreground service of type *location* requires the Play Console foreground-service declaration (a short video showing the logging/navigation feature).

15. Limitations and risks (read before production roll-out)

Risk	Impact	Mitigation
Not yet validated on a real underground ride	Sensor thresholds (stillness, vibration, departure acceleration) are estimates	Field test (Technical Design §15) and tune constants from the logs
Train cruising extremely smoothly could look “still” for ≥ 5 s	False station snap	Snap only within a window around the expected next station; review with field logs
Magnetic disturbance in trains	Wrong departure direction if the compass error > 90°	Tolerance of ±90°, user can pick destination or *Flip*; tested with 60° error
OpenStreetMap data gaps (missing stations, wrong order)	Wrong station names / distances	Stations filtered and ordered along the track; *Refresh lines*; data can be corrected in OSM
Overpass servers busy	Lines not downloaded	3 servers, back-off retries, retry on reconnect, whole-city pre-download
Phone manufacturers killing background apps	Logging stops with screen off	Foreground service + wake lock + battery-optimisation warning with *Fix*

Risk	Impact	Mitigation
Phone kept in a bag/pocket vs hand	Walking/vibration classification differences	Field-tune; step detector used when permitted

16. Glossary

Term	Meaning
GNSS	Global Navigation Satellite System (GPS, GLONASS, Galileo, BeiDou …)
Fused location	Google Play services location mixing GNSS, Wi-Fi and cell data
Dead reckoning (DR)	Estimating position from last known position, speed, heading and time
Track-constrained navigation	Estimating only the distance along a known railway line
ZUPT	Zero-velocity update: when the phone is still, speed is set to zero
Station snap	Setting the estimated position exactly to a station when the train stops there
Kalman update	Statistically weighted blend of an estimate and a measurement by their uncertainties
Overpass API	Read-only query service for OpenStreetMap data
Uncertainty (±m)	Estimated one-sigma position error shown with every position